

AXLERO SOLUTIONS

PROJECT DOCUMENT – ADVANCED MERN STACK ENGINEERING

Real-World Applications - Hands-on System Building

To ensure your MERN (MongoDB, Express, React, Node.js) portfolio stands out, we are moving away from standard CRUD applications (like To-Do lists or basic e-commerce sites). This volume focuses on **Real-Time Collaboration, Complex Event Sourcing, and AI-Augmented Workflows.**

Here are three highly advanced MERN stack projects for a senior-level portfolio.

Project 1 - "SyncSpace": Real-Time Collaborative Whiteboard & Code Editor

Domain: Developer Tools & Real-Time Collaboration

Problem Statement: Standard web applications operate on a request/response model. Building a system where multiple users can draw on a canvas or type code simultaneously without race conditions, lag, or data overwriting requires complex state synchronization algorithms (Operational Transformation or CRDTs) that standard MERN tutorials do not cover.

Use Case: A distributed engineering team uses SyncSpace for technical interviews. Candidate A draws an architecture diagram on the left side of the screen using the React canvas, while Interviewer B simultaneously writes Node.js code on the right side. The system uses WebSockets to broadcast changes instantly, and Conflict-free Replicated Data Types (CRDTs) to ensure that if both users edit the same line of code at the exact same millisecond, the final state merges perfectly without breaking the document.

Key Modules:

- **Real-Time Sync Engine (WebSockets/Socket.io):** Handles low-latency, bi-directional communication between clients.
- **CRDT Implementation (Yjs):** A framework integrated into React to handle complex, concurrent document editing and conflict resolution mathematically.
- **Interactive Canvas (React & Konva.js / Fabric.js):** A high-performance 2D canvas for drawing shapes, lines, and text.
- **Code Editor Integration (Monaco Editor):** Embedding Microsoft's Monaco (the engine behind VS Code) directly into the React frontend, synced via Yjs.

Week-wise Development Plan:

Week	Backend & Synchronization (Node, Express, Socket.io)	Frontend & Canvas (React, Yjs, Konva.js)
Week 1	Socket Infrastructure: Set up Express and Socket.io. Create "Rooms" so different teams can join isolated collaborative sessions.	UI Scaffolding: Build a React split-screen layout (Whiteboard on the left, Code Editor on the right).
Week 2	CRDT Integration (Backend): Configure the backend to act as a Yjs awareness server, broadcasting cursor positions and tracking connected users.	Canvas Engineering: Integrate Konva.js. Build tools for drawing freehand lines, rectangles, and text, syncing coordinates via WebSockets.
Mid-Project Review	Sync Audit: Open two browser tabs and prove that drawing a shape in Tab A instantly appears in Tab B.	Awareness Check: Ensure user cursors (with names) render smoothly across all connected clients.
Week 3	Persistence Layer: Connect Yjs to MongoDB. Periodically save the binary document state to the database so sessions survive server restarts.	Code Editor Sync: Integrate Monaco Editor. Bind the editor's text model directly to the Yjs shared string type for simultaneous editing.
Week 4	Access Control: Implement JWT authentication. Ensure users can only join rooms they have been invited to.	Refine & Polish: Add a "Replay" feature, allowing users to scrub backward through the session history to see how the code/drawing evolved.
Final Review	A masterclass in WebSockets, CRDTs, and real-time state synchronization.	A professional, dual-pane interactive IDE matching the complexity of modern SaaS tools.